

AASTool API Reference

AASTool — API Reference (OpenAPI-Style)

Enterprise Edition | Version 1.0.0

Base URL: `http://localhost:4000/api/v1`

Table of Contents

1. [Conventions](#)
 2. [Health](#)
 3. [Evaluation](#)
 4. [Assessments](#)
 5. [Buildings](#)
 6. [Criteria](#)
 7. [Building Types](#)
 8. [NEB Thresholds](#)
 9. [Configuration](#)
 10. [Disability Types & Dimensions](#)
 11. [Reports](#)
 12. [Authentication](#)
 13. [Error Reference](#)
-

Conventions

Response Envelope

All API responses follow a consistent envelope:

```
{  
  "ok": true,  
  "data": { ... }  
}
```

Error responses:

```
{
  "ok": false,
  "error": "Human-readable error message"
}
```

HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad request (validation error)
404	Resource not found
500	Internal server error
503	Service degraded (health check)

API Versioning

The current API version is v1 mounted at /api/v1/. All endpoints below are relative to <http://localhost:4000/api/v1>.

Health

GET /health (root)

Purpose: Infrastructure health check (outside API version path).

Response 200:

```
{
  "status": "ok",
  "uptime": 3600,
  "database": "connected",
  "dbLatencyMs": 2,
  "version": "1.0.0",
  "timestamp": "2026-06-15T12:00:00.000Z"
}
```

GET /api/v1/health

Purpose: API-scoped health check with envelope.

Response 200:

```
{
  "ok": true,
  "data": {
    "status": "ok",
    "uptime": 3600,
    "database": "connected",
    "dbLatencyMs": 2,
    "version": "1.0.0",
    "timestamp": "2026-06-15T12:00:00.000Z"
  }
}
```

Response 503 (database disconnected):

```
{
  "status": "degraded",
  "uptime": 3600,
  "database": "disconnected",
  "dbLatencyMs": null,
  "version": "1.0.0",
  "timestamp": "2026-06-15T12:00:00.000Z"
}
```

Evaluation

POST /api/v1/evaluate

Purpose: Run the accessibility evaluation algorithm. This is the core endpoint — all scoring, classification, and analytics happen here.

Request Body:

```
{
  "buildingType": "Commercial Buildings",
  "scores": {
    "EC1.1.1": 4,
    "EC1.1.2": 5,
    "EC1.2.1": 3
  }
}
```

Field	Type	Required	Default	Description
buildingType	string	No	"Commercial Buildings"	Building type name (must exist in DB)
scores	object	No	{}	Map of criterion code → score (1-5)

Validation: - Body must be a JSON object (not array, not scalar) - buildingType must be a string if provided - scores must be a plain object (Record<string, number>) if provided

Response 200:

```
{
  "ok": true,
  "result": {
    "obs": 72.45,
    "nebClass": "A",
    "nebEquivalent": "Very Good",
    "nebMeaning": "Building demonstrates strong
accessibility with minor areas for improvement",
    "nebScore": 4,
    "averageRawScore": 3.82,
    "avgRawScoreByDT": {
      "1": 4.1,
      "2": 3.5,
      "3": 3.9,
      "4": 3.8,
      "5": 3.8
    },
    "avgRawScoreByAD": {
      "1": 3.9,
      "2": 4.0,
      "3": 3.7,
      "4": 3.8,
      "5": 3.7
    },
    "strengths": [
      {
        "code": "EC1.1.2",
        "name": "Automated door systems",
        "score": 5,
        "is": 4.15
      }
    ],
    "weaknesses": [
      {
        "code": "EC2.3.1",
        "name": "Tactile floor indicators",
        "score": 1,
        "is": 0.83
      }
    ],
    "scoreDistribution": {
      "1": 3,
      "2": 8,
      "3": 22,
      "4": 18,
```

```
      "5": 12
    },
    "tisByDT": {
      "1": 85.5,
      "2": 72.3,
      "3": 81.0,
      "4": 79.2,
      "5": 79.2
    },
    "cisByAD": {
      "1": 397.25,
      "2": 408.5,
      "3": 378.1,
      "4": 388.3,
      "5": 378.1
    },
    "criteria": [
      {
        "code": "EC1.1.1",
        "name": "...",
        "score": 4,
        "is": 3.32
      }
    ]
  }
}
```

Response 400:

```
{ "ok": false, "error": "Request body must be a JSON
object" }
{ "ok": false, "error": "buildingType must be a string" }
{ "ok": false, "error": "scores must be an object mapping
codes to numbers" }
```

Response 500:

```
{ "ok": false, "error": "Internal error" }
```

Assessments

POST /api/v1/assessments

Purpose: Create a new assessment (runs evaluation and saves the result to the database).

Request Body:

```
{
  "buildingId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Office Tower Audit Q3 2026"
}
```

Field	Type	Required	Description
buildingId	string	Yes	UUID of the BuildingType
name	string	No	Optional label for the assessment

Response 201:

```
{
  "ok": true,
  "data": {
    "id": "660e8400-e29b-41d4-a716-446655440001",
    "buildingId": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Assessment - Commercial Buildings",
    "results": {
      "obs": 72.45,
      "nebClass": "A",
      "nebEquivalent": "Very Good",
      "...": "..."
    }
  }
}
```

Response 500:

```
{ "ok": false, "error": "Failed to create assessment" }
```

GET /api/v1/assessments/:id

Purpose: Retrieve a saved assessment by ID.

Path Parameters:

Param	Type	Description
id	string	UUID of the Evaluation

Response 200:

```
{
  "ok": true,
  "data": {
    "id": "660e8400-e29b-41d4-a716-446655440001",
    "buildingId": "550e8400-e29b-41d4-a716-446655440000",
    "results": {
      "obs": 72.45,
      "nebClass": "A",
      "...": "..."
    }
  }
}
```

```
}  
}  
}
```

Response 404:

```
{ "ok": false, "error": "Assessment not found" }
```

Buildings

GET /api/v1/buildings

Purpose: List all building types with their assessment history.

Response 200:

```
{  
  "ok": true,  
  "data": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000",  
      "name": "Commercial Buildings",  
      "type": "Commercial Buildings",  
      "assessments": [  
        {  
          "id": "660e8400-...",  
          "status": "Completed",  
          "created_at": "2026-06-15T12:00:00.000Z"  
        }  
      ]  
    }  
  ]  
}
```

GET /api/v1/buildings/:id

Purpose: Get building type detail with full assessment history.

Path Parameters:

Param	Type	Description
id	string	UUID of the BuildingType

Response 200: Same structure as above but single object.

Response 404:

```
{ "ok": false, "error": "Building not found" }
```

Criteria

GET /api/v1/criteria

Purpose: List all evaluation criteria (63 total).

Response 200:

```
{
  "ok": true,
  "data": [
    {
      "id": "770e8400-...",
      "code": "EC1.1.1",
      "name": "Automated door width compliance",
      "definition": "Doors must have a clear opening width
of at least 850mm...",
      "justification": "Wheelchair users require sufficient
clearance...",
      "value": 0.83,
      "disability": 1,
      "dimension": 1,
      "score": 3,
      "levels": [
        "Level 1: Manual doors, no automation",
        "Level 2: Manual doors with accessible hardware",
        "Level 3: Semi-automatic push-button doors",
        "Level 4: Fully automatic sliding doors",
        "Level 5: Smart doors with proximity sensors and
voice control"
      ]
    }
  ]
}
```

Sort order: Ascending by code.

POST /api/v1/criteria

Purpose: Create a new evaluation criterion.

Request Body:

```
{
  "code": "EC1.1.10",
  "name": "New criterion name",
  "definition": "Detailed definition text",
  "justification": "Why this matters (optional)",
}
```



```
"value": 0.75,  
"disability": 1,  
"dimension": 1,  
"levels": [  
  "Level 1 description",  
  "Level 2 description",  
  "Level 3 description",  
  "Level 4 description",  
  "Level 5 description"  
]  
}
```

Field	Type	Required	Description
code	string	Yes	Must match /^EC\d+\.\d+\.\d+\$/
name	string	Yes	Short name
definition	string	Yes	Full definition
justification	string	No	Why this criterion matters
value	number	No	Base weight (float)
disability	number	No	DT ID (1-5)
dimension	number	No	AD ID (1-5)
levels	string[]	Yes	Exactly 5 level descriptions

Validation Errors (400):

```
{ "ok": false, "error": "code, name, and definition are  
required" }  
{ "ok": false, "error": "code must match pattern ECx.y.z  
(e.g. EC1.1.3)" }  
{ "ok": false, "error": "levels must be an array of exactly  
5 strings" }  
{ "ok": false, "error": "disability must be between 1 and  
5" }
```

Response 201:

```
{  
  "ok": true,  
  "data": {  
    "id": "880e8400-...",  
    "code": "EC1.1.10",  
    "name": "New criterion name",  
    "...": "..."  
  }  
}
```

GET /api/v1/criteria/:code

Purpose: Get a single criterion by its code.

Path Parameters:

Param	Type	Description
code	string	Criterion code, e.g. EC1.1.1

Response 200: Single criterion object (same shape as list item).

Response 404:

```
{ "ok": false, "error": "Criterion not found" }
```

Building Types

GET /api/v1/building-types

Purpose: List all building type definitions with their disability and dimension weights.

Response 200:

```
{
  "ok": true,
  "data": [
    {
      "id": "550e8400-...",
      "name": "Commercial Buildings",
      "disability_weights": [1.0, 0.8, 1.2, 1.0, 0.9],
      "dimension_weights": [1.0, 1.0, 1.0, 1.0, 1.0]
    }
  ]
}
```

Sort order: Ascending by name.

NEB Thresholds

GET /api/v1/neb-thresholds

Purpose: Get NEB (Normalized Efficiency Band) classification thresholds — the mapping from OBS ranges to letter grades.

Response 200:

```
{
  "ok": true,
  "data": [
    {
      "id": "990e8400-...",
      "min": 80,
      "neb_class": "A+",
      "equivalent": "Excellent",
      "meaning": "The building demonstrates outstanding
accessibility across all disability types and dimensions.",
      "neb_score": 5,
      "ordinal": 1
    },
    {
      "min": 65,
      "neb_class": "A",
      "equivalent": "Very Good",
      "meaning": "Strong accessibility with minor areas for
improvement.",
      "neb_score": 4,
      "ordinal": 2
    }
  ]
}
```

Sort order: Descending by `min` (highest threshold first). The last band (D) has `min`: 0.

OBS ≥	Class	Score
80	A+	5
65	A	4
50	B	3
35	C	2
0	D	1

Configuration

GET /api/v1/config

Purpose: Retrieve all system configuration parameters from the database.

Response 200:

```
{
  "ok": true,
  "data": [
```

```
{
  "id": "aa0e8400-...",
  "key": "MAX_OBS",
  "value": "1000"
},
{
  "id": "aa0e8401-...",
  "key": "DEFAULT_SCORE",
  "value": "3"
},
{
  "id": "aa0e8402-...",
  "key": "STRENGTH_THRESHOLD",
  "value": "5"
},
{
  "id": "aa0e8403-...",
  "key": "WEAKNESS_THRESHOLD",
  "value": "2"
},
{
  "id": "aa0e8404-...",
  "key": "TOP_N_RESULTS",
  "value": "5"
}
]
```

Key	Purpose
MAX_OBS	Normalization divisor for OBS calculation
DEFAULT_SCORE	Fallback score when no user score provided
STRENGTH_THRESHOLD	Minimum score to highlight as strength
WEAKNESS_THRESHOLD	Maximum score to highlight as weakness
TOP_N_RESULTS	Number of top strengths/weaknesses returned

Disability Types & Dimensions

GET /api/v1/disability-types

Purpose: Get the list of disability type labels (DT1-DT5).

Response 200:

```
{
  "ok": true,
  "data": [
```

```

        { "id": 1, "name": "Physical", "description": "DT 1 –
Physical" },
        { "id": 2, "name": "Sensory", "description": "DT 2 –
Sensory" },
        { "id": 3, "name": "Cognitive", "description": "DT 3 –
Cognitive" },
        { "id": 4, "name": "Intellectual", "description": "DT 4
– Intellectual" },
        { "id": 5, "name": "Psychosocial", "description": "DT 5
– Psychosocial" }
    ]
}

```

Data source: Loaded from building-types.json data file. Labels are parsed to extract the human-readable name after the “DT N —” prefix.

GET /api/v1/assessment-dimensions

Purpose: Get the list of assessment dimension labels (AD1-AD5).

Response 200:

```

{
    "ok": true,
    "data": [
        { "id": 1, "name": "Wayfinding", "description": "AD 1 –
Wayfinding" },
        { "id": 2, "name": "Entry", "description": "AD 2 –
Entry" },
        { "id": 3, "name": "Circulation", "description": "AD 3
– Circulation" },
        { "id": 4, "name": "Facilities", "description": "AD 4 –
Facilities" },
        { "id": 5, "name": "Evacuation", "description": "AD 5 –
Evacuation" }
    ]
}

```

Reports

GET /api/v1/reports

Purpose: List all generated evaluation reports with human-readable titles.

Response 200:

```
{
  "ok": true,
  "data": [
    {
      "id": "660e8400-...",
      "title": "Accessibility Report #1 – Commercial
Buildings (Jun 15, 2026) – OBS: 72.45%"
    }
  ]
}
```

Sort order: Descending by `created_at` (most recent first).

Title format: Accessibility Report #N – {building_type} ({date}) – OBS: {obs}%

Where N counts from 1 (oldest) to the total number of evaluations.

Authentication

POST `/api/v1/login`

Purpose: User login. **Generates real HMAC-SHA256 tokens via Node.js crypto.**

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Field	Type	Required	Validation
email	string	Yes	Must contain @
password	string	Yes	Minimum 6 characters

Configuration (environment variables):

Variable	Default	Description
JWT_SECRET	<i>(required — 500 if missing)</i>	HMAC secret key
JWT_EXPIRES_IN	3600	Token lifetime in seconds
LOGIN_REDIRECT	/dashboard	Post-login redirect path

Response 200:

```
{
  "ok": true,
  "accessToken": "<48-char-hex-hmac>",
  "refreshToken": "<48-char-hex-hmac>",
  "expiresIn": 3600,
  "redirectTo": "/dashboard"
}
```

Response 500 (missing secret):

```
{ "ok": false, "error": "Server configuration error:
JWT_SECRET not set" }
```

Response 400:

```
{ "ok": false, "error": "Email and password are required" }
{ "ok": false, "error": "Invalid email address" }
{ "ok": false, "error": "Password must be at least 6
characters" }
```

Error Reference

Standard Error Format

All errors follow this structure:

```
{
  "ok": false,
  "error": "Human-readable description of what went wrong"
}
```

Error Catalog

Status	Error Message	Endpoint(s)	Cause
400	Invalid JSON body	All (middleware)	Malformed JSON in request body
400	Malformed JSON	All (middleware)	Syntax error in JSON body
400	Request body must be a JSON object	POST /evaluate	Body is array or scalar
400	buildingType must be a string	POST /evaluate	Wrong type

Status	Error Message	Endpoint(s)	Cause
400	scores must be an object mapping codes to numbers	POST /evaluate	Wrong type
400	code, name, and definition are required	POST /criteria	Missing required field
400	code must match pattern ECx.y.z	POST /criteria	Invalid code format
400	levels must be an array of exactly 5 strings	POST /criteria	Wrong levels count
400	disability must be between 1 and 5	POST /criteria	Out of range
400	Email and password are required	POST /login	Missing credentials
400	Invalid email address	POST /login	No @ in email
400	Password must be at least 6 characters	POST /login	Too short
404	Criterion not found	GET /criteria/:code	Invalid code
404	Building not found	GET /buildings/:id	Invalid UUID
404	Assessment not found	GET /assessments/:id	Invalid UUID
500	Internal error	POST /evaluate	Unexpected server error
500	Failed to create assessment	POST /assessments	DB/save failure
500	Failed to fetch assessment	GET /assessments/:id	DB/query failure
500	Failed to fetch buildings	GET /buildings	DB/query failure
500	Failed to fetch building details	GET /buildings/:id	DB/query failure
500	Failed to fetch reports	GET /reports	DB/query failure
500	Internal server error during login	POST /login	Unexpected error
500	failed	GET /criteria	DB/query failure

Rate Limiting

Rate limiting is **not implemented** in the current version. The backend relies on Express's default behavior with no per-IP or per-endpoint throttling. Add `express-rate-limit` middleware for production.

CORS

CORS is enabled for all origins via `app.use(cors())`. Restrict this in production to only the frontend's domain.

AASTool by Serg | Dev by Y